```
    @Test
    public void should_transfer_funds_to_a_different_bank() {...}
    ...

    @Test
    public void should_deduct_fees_as_a_separate_transaction() {...}
    ...
}
```

Tests that are written this way read more like specifications than unit tests. They focus on the behavior of the application, using tests simply as a means to express and verify that behavior. North also noted that tests written this way are much easier to maintain because their intent is so clear. The impact of this approach was so significant that he started referring to what he was doing no longer as Test-Driven Development, but as *Behavior*-Driven Development.

1.3.2 BDD also works well for requirements analysis

But describing a system's behavior turns out to be what business analysts do every day. Working with business analyst colleague Chris Matts, North set out to apply what he had learned to the requirements-analysis space. Around this time, Eric Evans introduced the idea of Domain-Driven Design,[13] which promotes the use of a ubiquitous language that business people can understand to describe and model a system. North and Matts's vision was to create a ubiquitous language that business analysts could use to define requirements unambiguously, and that could also be easily transformed into automated acceptance tests. To implement this vision, they started expressing the acceptance criteria for user stories in the form of loosely structured examples, known as "scenarios," like this one:

```
Given a customer has a current account
When the customer transfers funds from this account to an overseas account
Then the funds should be deposited in the overseas account
And the transaction fee should be deducted from the current account
```

A business owner can easily understand a scenario written like this. It gives clear and objective goals for each story in terms of what needs to be developed and of what needs to be tested.

This notation eventually evolved into a commonly used form often referred to as Gherkin. With appropriate tools, scenarios written in this form can be turned into automated acceptance criteria that can be executed automatically whenever required. Dan North wrote the first dedicated BDD test automation library, JBehave, in the mid-2000s, and since then many others have emerged for different languages, both at the unit-testing and acceptance-testing levels.

[13] Eric Evans, *Domain Driven Design* (Addison-Wesley Professional, 2003).